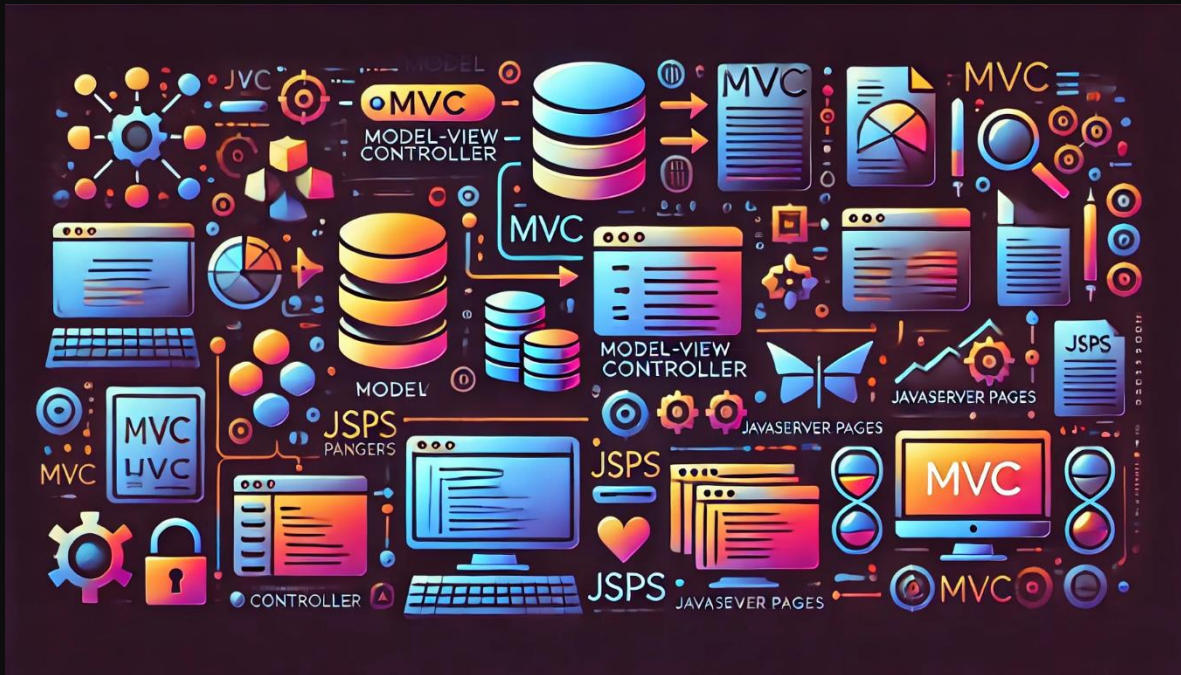


# MVC con JSPs - Ejemplo 2



## Guía para crear un Proyecto con MVC, Servlets y JSPs – parte 2

Este proyecto es la continuación del proyecto anterior, donde implementamos el patrón de diseño MVC en una aplicación Java Web. La aplicación incluye un servlet controlador, una clase de modelo (`Rectangulo`) para representar los datos, y páginas JSP para mostrar información según diferentes alcances (`request`, `session`, y `application`). El flujo de la aplicación permite al usuario ver y administrar variables en cada uno de estos contextos.

## Paso 1: Crear el Proyecto en NetBeans

1. Abre **NetBeans** y selecciona Archivo -> Nuevo Proyecto.
2. Escoge Java Web -> Aplicación Web.
3. **Nombre del Proyecto:** "Ejemplo2MVC".
4. **Ubicación del Proyecto:** Elige una ubicación adecuada en tu sistema.
5. **Servidor:** Selecciona GlassFish como el servidor de despliegue.
6. **Framework:** Selecciona Jakarta EE para habilitar el soporte de servlets y JSPs.

## Paso 2: Crear la Clase del Modelo (`Rectangulo.java`)

1. En la estructura de paquetes del proyecto, crea un paquete llamado `modelo`.
2. En el paquete `modelo`, crea la clase `Rectangulo` con el siguiente código:
3. Esta es la misma clase del proyecto anterior.

```
package modelo;
```

```
public class Rectangulo {
    private int base;
    private int altura;

    public Rectangulo() {}

    public Rectangulo(int base, int altura) {
        this.base = base;
        this.altura = altura;
    }

    public int getBase() {
        return base;
    }

    public void setBase(int base) {
        this.base = base;
    }

    public int getAltura() {
        return altura;
    }

    public void setAltura(int altura) {
        this.altura = altura;
    }

    public int getArea() {
        return this.base * this.altura;
    }
}
```

## Paso 3: Crear el Archivo `index.jsp`

En el directorio `webapp`, crea el archivo `index.jsp` con el siguiente código:

```
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Ejemplo MVC</title>
    </head>
    <body>
```

```
<h1>Ejemplo MVC</h1>
<br/>
<div style="color:red">${mensaje}</div>
<br/>
<a href="${pageContext.request.contextPath}/ServletControlador">
    Link al servlet controlador SIN parámetros
</a>
<br/><br/>
<a
href="${pageContext.request.contextPath}/ServletControlador?accion=agregarVariables">
    Link al servlet controlador para agregar las variables
</a>
<br/><br/>
<a
href="${pageContext.request.contextPath}/ServletControlador?accion=listarVariables">
    Link al servlet controlador para listar variables
</a>
</body>
</html>
```

## Explicación de `index.jsp`

Este archivo es la página de inicio y contiene enlaces para llamar al servlet controlador con diferentes parámetros (`accion`). También muestra un mensaje informativo que puede ser agregado por el servlet.

## Paso 4: Crear el Servlet Controlador (`ServletControlador.java`)

1. En la estructura de paquetes del proyecto, crea un paquete llamado `controlador`.
2. En el paquete `controlador`, crea el servlet `ServletControlador` con el siguiente código:

```
package controlador;

import java.io.IOException;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import modelo.Rectangulo;
```

```
@WebServlet("/ServletControlador")
public class ServletControlador extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // 1. Procesamos parámetros
        String accion = request.getParameter("accion");

        // 2. Creamos los JavaBeans (clases de Modelo)
        Rectangulo rectanguloRequest = new Rectangulo(1, 2);
        Rectangulo rectanguloSession = new Rectangulo(3,4);
        Rectangulo rectanguloApplication = new Rectangulo(5,6);

        // 3. Agregamos el JavaBean a algún alcance (request, session, application)
        if("agregarVariables".equals(accion)){
            // Alcance request
            request.setAttribute("rectanguloRequest", rectanguloRequest);
            // Alcance session
            HttpSession session = request.getSession();
            session.setAttribute("rectanguloSession", rectanguloSession);
            // Alcance application
            ServletContext application = this.getServletContext();
            application.setAttribute("rectanguloApplication", rectanguloApplication);

            // Agregamos un mensaje
            request.setAttribute("mensaje", "Las variables fueron agregadas");
            // 4. Redireccionamos al jsp de index
            request.getRequestDispatcher("index.jsp").forward(request, response);
        }
        else if("listarVariables".equals(accion)){
            // Redirigimos al jsp que despliega las variables
            request.getRequestDispatcher("WEB-INF/alcanceVariables.jsp").forward(request,
response);
        }
        else{
            // Redirige a la página de inicio
            request.setAttribute("mensaje", "Acción no proporcionada o desconocida");
            request.getRequestDispatcher("index.jsp").forward(request, response);
        }
    }
}
```

## Explicación de `ServletControlador.java`

Este servlet:

- Procesa el parámetro `accion` para decidir qué acción realizar.
- Crea tres objetos `Rectangulo` para diferentes alcances (`request`, `session`, y `application`).
- Maneja dos opciones:
  - `agregarVariables`: Agrega las variables en los alcances especificados y redirige a `index.jsp`.
  - `listarVariables`: Muestra los datos de las variables en `alcanceVariables.jsp`.
- Redirige a `index.jsp` en caso de una acción desconocida.

## Paso 5: Crear la Vista `alcanceVariables.jsp`

Dentro de la carpeta `WEB-INF`, crea el archivo `alcanceVariables.jsp` con el siguiente código:

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Alcance de Variables</title>
  </head>
  <body>
    <h1>Alcance de Variables</h1>
    <br/>
    Variable request:
    Base: ${rectanguloRequest.base}
    Altura: ${rectanguloRequest.altura}
    Área: ${rectanguloRequest.area}
    <br/><br/>
    Variable session:
    Base: ${rectanguloSession.base}
    Altura: ${rectanguloSession.altura}
    Área: ${rectanguloSession.area}
    <br/><br/>
    Variable application:
    Base: ${rectanguloApplication.base}
    Altura: ${rectanguloApplication.altura}
    Área: ${rectanguloApplication.area}
    <br/><br/>
    <a href="${pageContext.request.contextPath}/index.jsp">Regresar al inicio</a>
  </body>
```

```
</html>
```

## Explicación de `alcanceVariables.jsp`

Esta vista muestra los valores de los objetos `Rectangulo` almacenados en diferentes alcances (`request`, `session`, y `application`), y permite regresar a la página de inicio.

## Paso 6: Ejecutar el Proyecto

1. Guarda todos los archivos y compila el proyecto.
2. Inicia GlassFish desde NetBeans.
3. Haz clic derecho sobre el proyecto y selecciona `Ejecutar`.
4. La aplicación abrirá `index.jsp` en el navegador, permitiendo probar los diferentes enlaces y ver las variables en los diferentes alcances.

## Conclusión

Este proyecto ilustra cómo utilizar alcances (`request`, `session`, `application`) con el patrón de diseño MVC, permitiendo al controlador administrar los datos y las vistas presentar los resultados. Esta estructura organizada facilita la administración de datos y la separación de responsabilidades en aplicaciones web.

Saludos!

**Ing. Ubaldo Acosta**

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)